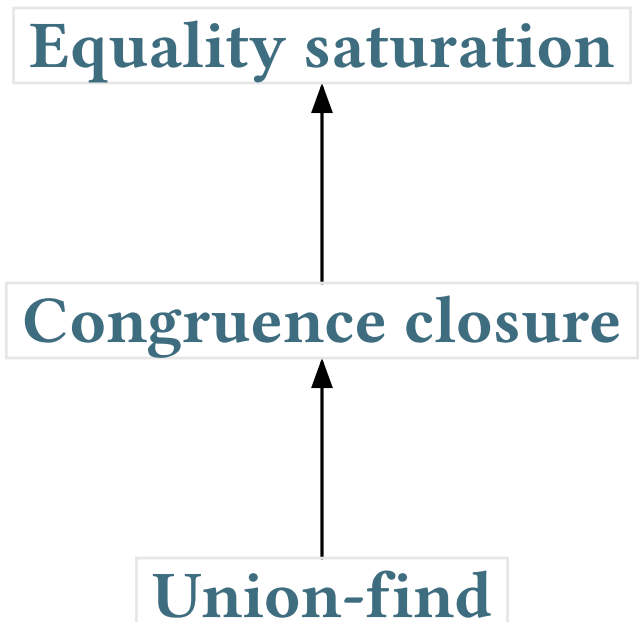
E-graphs modulo theories

presented by Sofia Brookie

E-graphs overview



E-graphs overview

Equality saturation

Handles non-ground equations (identities): $\forall x. f(x) = g(x, x)$

Congruence closure

Handles ground non-atomic equations: $f(e_1, e_2) = e_3$

Union-find

Handles ground atomic equations: $e_1 = e_2$

E-graphs vs AC

A: associativity

$$f(x, f(y, z)) = f(f(x, y), z)$$

C: commutativity

$$f(x, y) = f(y, x)$$

AC: both

E-graphs vs AC

The bane of equality saturation is operators that are AC or similar (ACU, abelian groups, etc.).

For simplicity, we will use ‘+’ as a generic name for an AC operator.

E-graphs vs AC

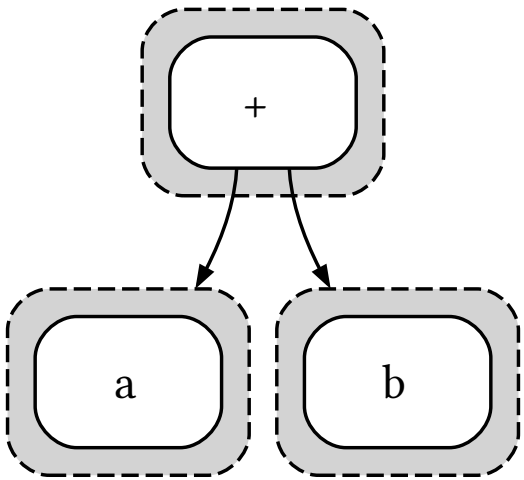


Figure 1: Size 2

E-graphs vs AC

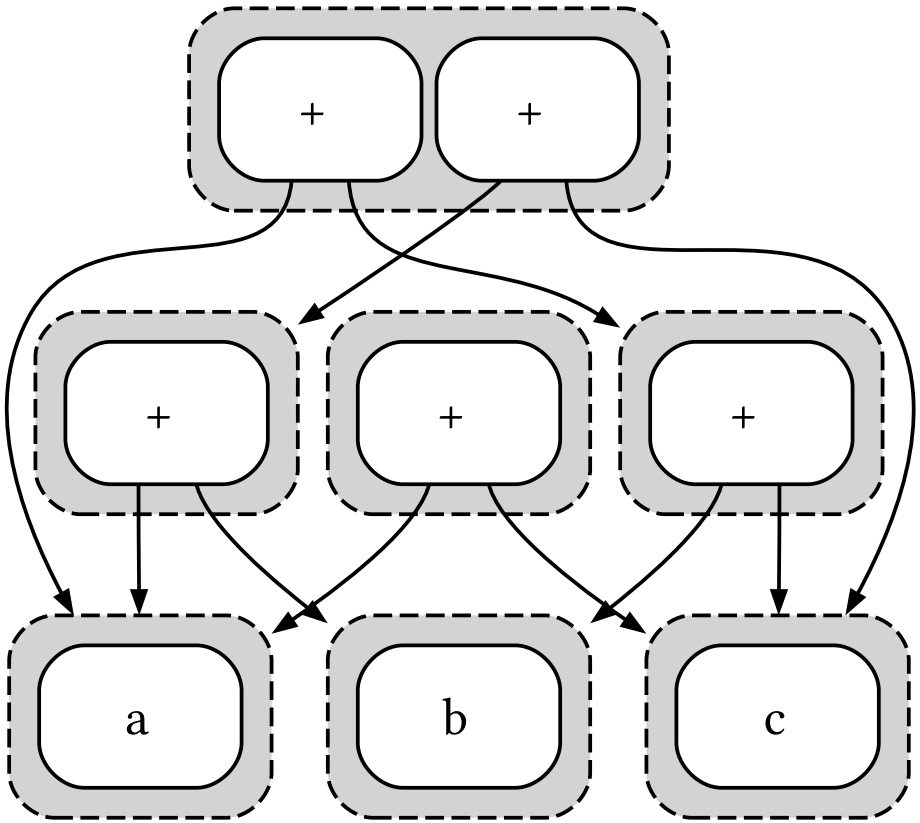


Figure 2: Size 3

E-graphs vs AC

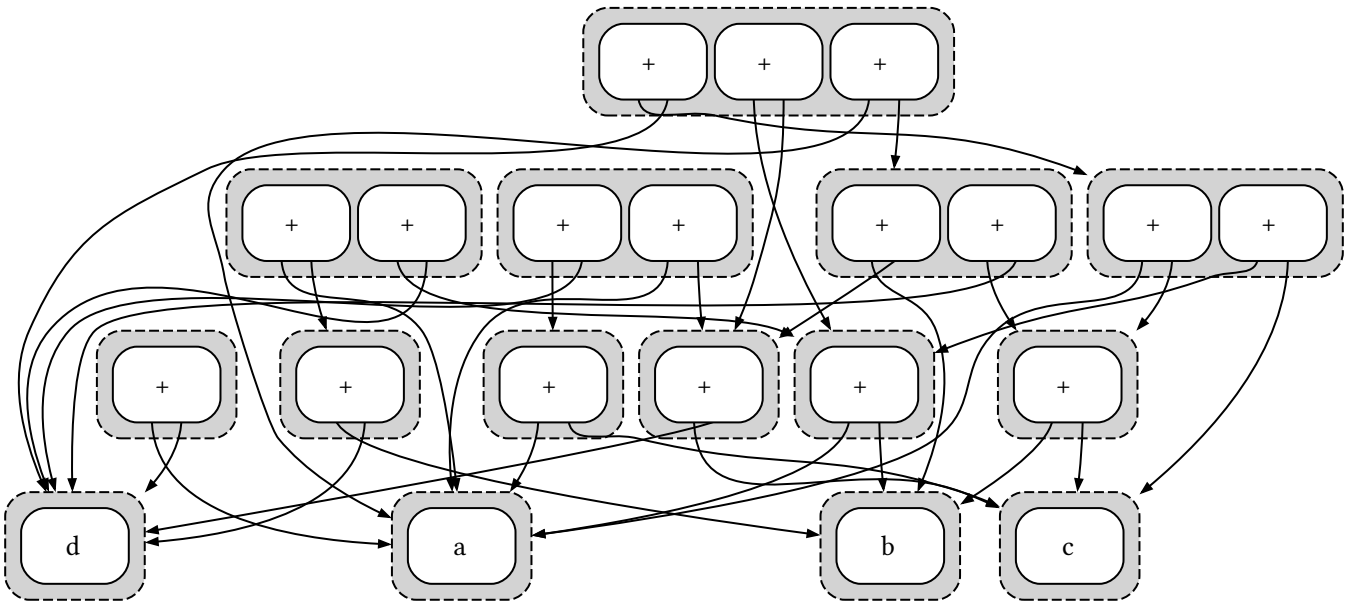


Figure 3: Size 4

E-graphs vs AC

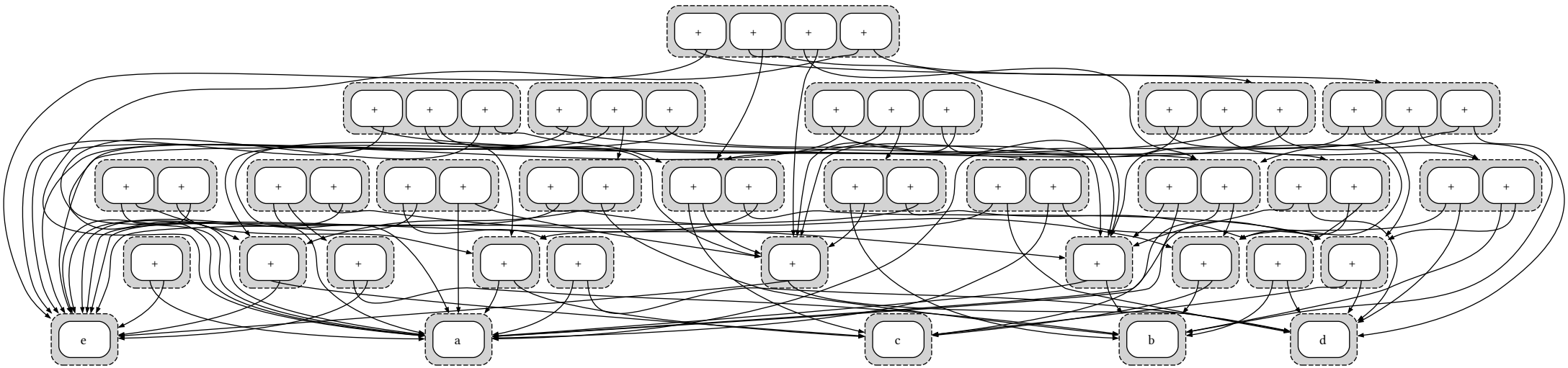


Figure 4: Size 5

E-graphs vs AC

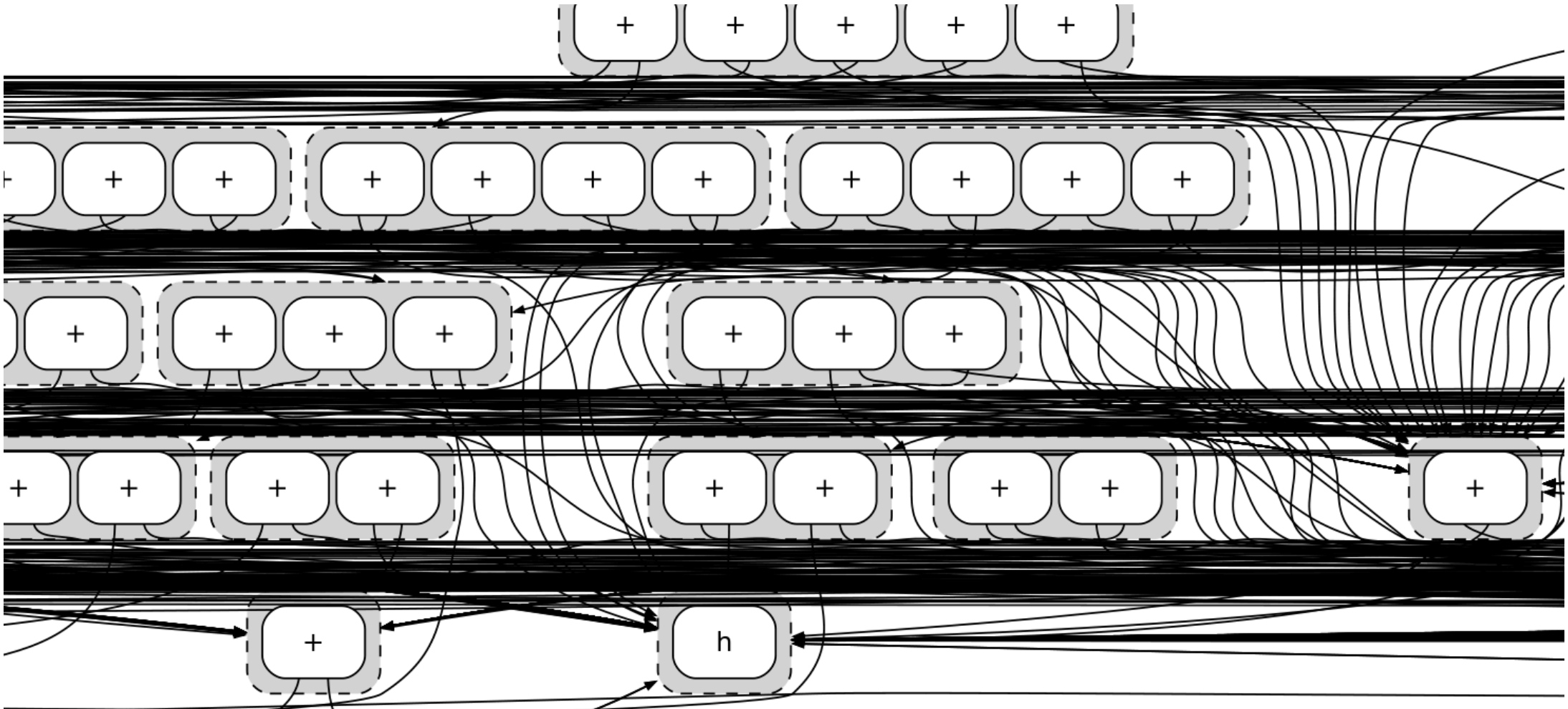


Figure 5: Size 8 (a small window)

E-graphs vs AC

Blindly saturating with A and C gives $O(2^n)$ E-classes if you have a starting term with n +s ($a + b + c + \ldots$), and each E-class has $O(2^n)$ E-nodes in it! So, **doubly exponential**.

Sidenote: What about just A?

A is ‘only’ cubic if you have no ‘loops’. Otherwise, it is $O(\infty)$. But making progress here is hard, given that A is fundamentally undecidable.

(Non)-solution

Backoff methods: fire rules less and less if they are used too many times

(Non)-solution

Backoff methods: fire rules less and less if they are used too many times

Issues:

- Doesn't help if you run to saturation
- Can slow down finding the equations you want to find

Non-solution

Consider an affine function f with identity

$$\forall xy.f(x+y)+1=f(x)+f(y)$$

and the starting term $f(a)+f(b)+f(c)$. We can to rewrite this to $f(a+b+c)+2$

E-graphs vs AC

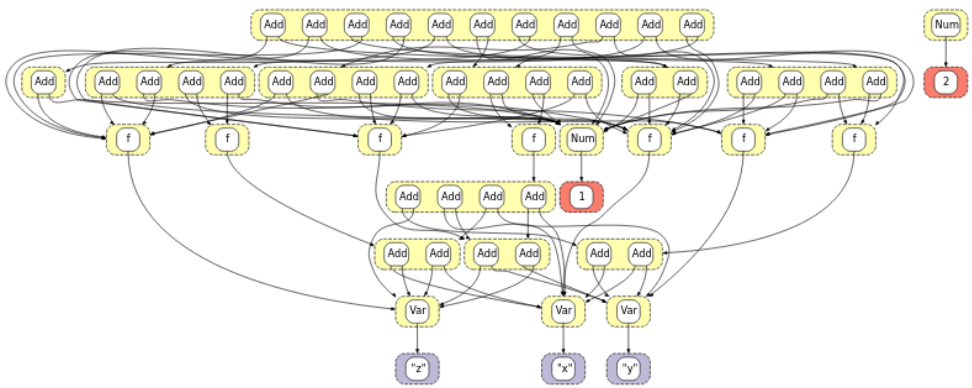


Figure 6: Size 4 (timeout)

The solution? EMTs

The SAT people have added theory-specific solvers and made very successful SMT solvers. Why can't we handle AC in a specific way, rather than relying on equality saturation?

The solution? EMTs

The SAT people have added theory-specific solvers and made very successful SMT solvers. Why can't we handle AC in a specific way, rather than relying on equality saturation?

EMTs seems to be an idea thrown around for a while, in the hope that someone will eventually make this natural thing work. Unfortunately, it is not as easy as has been suggested.

EMT example

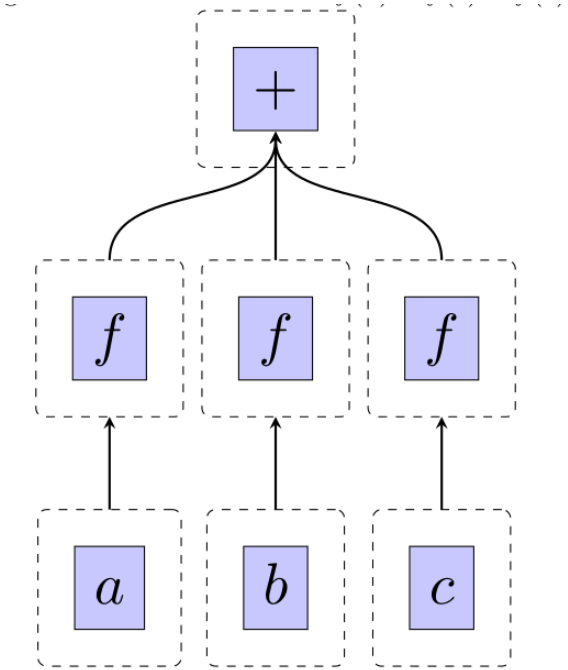


Figure 7: EMTs to the rescue…

EMT example

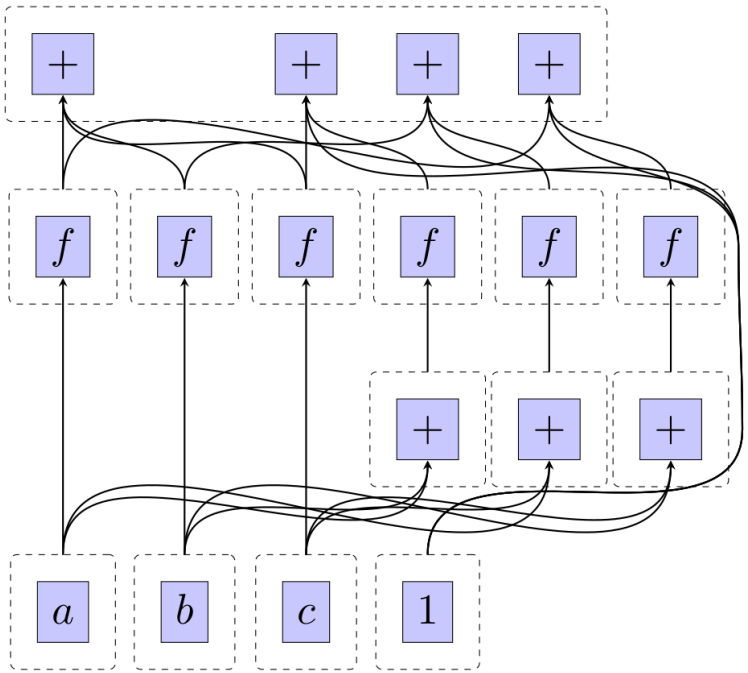


Figure 8: EMTs to the rescue…

The solution??

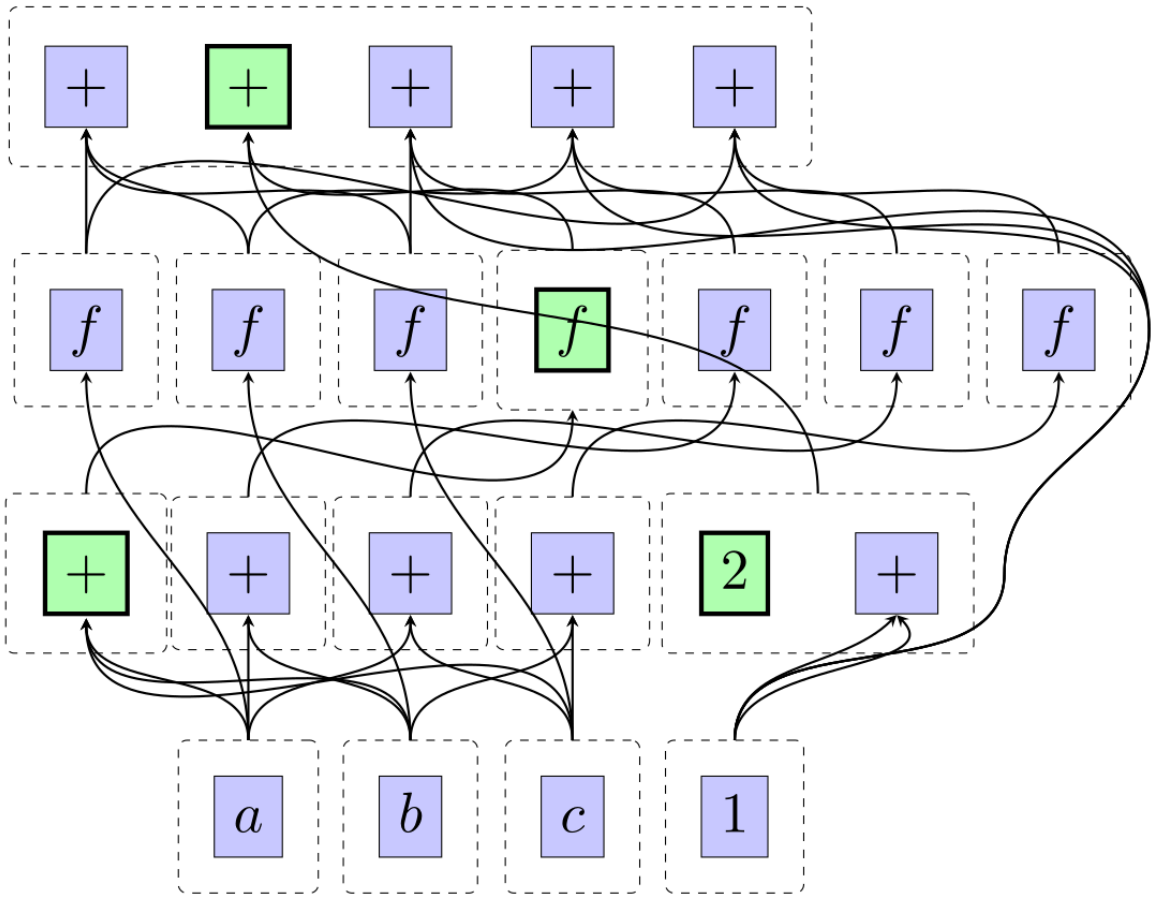


Figure 9: EMTs to the rescue...

E-graphs as rewrite systems

E-graphs connect very closely to the theory of **rewriting**.

The 'Abstract Congruence Closure' perspective is that E-graphs are building a *convergent term rewriting system* on an extended signature.

Congruence closure is very similar to 'ground Knuth-Bendix completion' from rewriting.

(Equality saturation is a bit of a stranger animal but still vaguely completion-flavoured)

E-graphs as tree automata

E-graphs are reachable deterministic bottom-up tree automata

- Reachable: every E-class is inhabited by some E-node
- Deterministic: if $f(s_1, s_2) \to s_3$ and $f(s_1, s_2) \to s_4$ then $s_3 = s_4$
- Bottom-up: there is a notion of top-down deterministic tree automata but it is weaker than deterministic bottom-up (compare the string automata case, where left-to-right and right-to-left are equivalent)

Inspirations for adding the “mod T”

Rewriting modulo theories

Algebraic automata (that is, tree automata on non-free theories)

The neat thing about E-graphs

...is that they add a new symbol for everything. $f(e_1)$? Well, if we haven't seen that before, give it a new E-class/tree-automata state

Confluence

If we have a term that is ‘in multiple E-classes’, we ought to merge the E-classes.

This is what rebuilding does:

$e_1 \to e_2$ and $f(\ldots e_1 \ldots) \to e_3$ (‘parent-child overlap’)

$f(\ldots e_1 \ldots) \to e_3$ and $f(\ldots e_1 \ldots) \to e_4$ (‘self-overlap’)

The new symbols for E-classes means we can handle granchild overlap, etc, by iterating these basic steps.

...which is how they simplify confluence

If we have a critical pair of terms, they must have some relevant 'overlap' between rules. Any overlap must be a case of subterm containment. Every subterm gets a new symbol, so we only have to check for a few cases:

$e_1 \to e_2$ and $f(...e_1...) \to e_3$ ('parent-child overlap')

$f(...e_1...) \to e_3$ and $f(...e_1...) \to e_4$ ('self-overlap')

Grandchild overlap, etc, is handled by fixpoint propagation.

Key notion: critical pairs

A critical pair is some minimal term that can rewrite in two ways/is in two E-classes. To do congruence closure (modulo T) properly, we need to find critical pairs and resolve them, by merging the E-classes.

AC-critical pairs

$$(a + b) + c = e_1$$
$$b + (c + d) = e_2$$

$b + c$ is the *overlap* of an AC-critical pair:

$$e_1 + d \underset{AC}{=} a + b + c + d \underset{AC}{=} a + e_2$$

What are we doing in AC equality saturation? Well, any of the $O(2^n)$ AC-subterms *could* become an AC-critical pair. We are eagerly giving new E-classes to them. Just in case. After all, equality saturation only

works by grounding identities: we have not given a mechanism to ground something only when it's actually needed.

EMTs, prehistorically

Existing suggestions for EM(T)s fall along two very similar lines:

- Canonize the T parts of the E-graph, and work mod T in some way
- Treat E-nodes as 'containers' (of E-class names), and move from fixed-arity function symbols to lists, multisets/bags, etc

Either way is an attempt to bake the theories in:

- A(U) = semigroups = lists = associate to the right
- AC(U) = commutative semigroups = multisets/bags = sort ascending and associate to the right

EMTs, the end of prehistory

The intuition:

‘Why do we need all these AC-equivalent nodes? Can’t we just pick an AC-representative?‘

Yes, *but*

You need some method of finding T-critical pairs, and to make a complete algorithm, you must *eventually* resolve every pair.

And also match modulo T to instantiate identities for equality saturation.

Counterexamples to prehistoric EMTs

$$a + b = a$$

Modulo A, we ought to generate the **infinite** set of implied subterms $b, b + b, \ldots$. So, prehistoric methods fail when E-graphs are **not flat terms**.

Counterexamples to prehistoric EMTs

$$b + a + c = a$$

The infinite set of equivalent terms modulo A is now $nb + a + nc$. We can no longer recognize the set of all re-associations of this, or the set of canonized forms of this, with a tree automata.

Counterexamples to prehistoric EMTs

$$a + (b + c) = d$$

$$a + b = e$$

$$e + c = f$$

These equations are individually canonical, but they don't include the consequence $d = a + (b + c) = (a + b) + c = e + c = f$

Prehistoric methods might canonize, but this means they don't scan for implied equalities among a set of (canonized!) equations. Actually...

EMTs and the word problem

Scanning for implied equalities is exactly the same thing as resolving all T-critical pairs

Any mechanism to automatically resolve all T-critical pairs is a solver for the T-word problem.

A and AC

Finding A-critical pairs, and matching modulo A, is quite feasible. Still, there is not ‘packaged’ solution possible: the A-word problem is undecidable. So, the best we can do is just provide a list of critical pairs for the ‘outer loop’ of equality saturation to choose to resolve (or not)

AC is better: the word problem is decidable (Mayr-Meyer). It’s ESPACE-complete… Sounds bad, but E-graphs are EESPACE here, but more importantly: we can solve common cases of the AC word problem faster than ESPACE (hopefully), whereas E-graphs *must* chug along to

saturation to be *certain* (as they have no smart way of knowing if some AC-subterm is relevant to the critical pair problem)

The AC word problem, a la Bachmair et al.

Given $a_1 + a_2 + \ldots \to e_1$ and $b_1 + b_2 + \ldots \to e_2$, construct the potential critical pair

$$e_1 + \ldots = a_1 + a_2 + \ldots + b_1 + b_2 = \ldots + e_2$$

Reduce it by existing rules, orient it by some (multiset) ordering and add it as a rule if it is non-trivial.

The AC word problem, a la Bachmair et al.

Given $a_1 + a_2 + \ldots \to e_1$ and $b_1 + b_2 + \ldots \to e_2$, construct the potential critical pair

$$e_1 + \ldots = a_1 + a_2 + \ldots + b_1 + b_2 = \ldots + e_2$$

Reduce it by existing rules, orient it by some (multiset) ordering and add it as a rule if it is non-trivial.

Very similar to Buchberger's algorithm (see later). Note: we use + in our examples, but this will be written as × when we embed our terms into polynomials!

The AC word problem, brief example

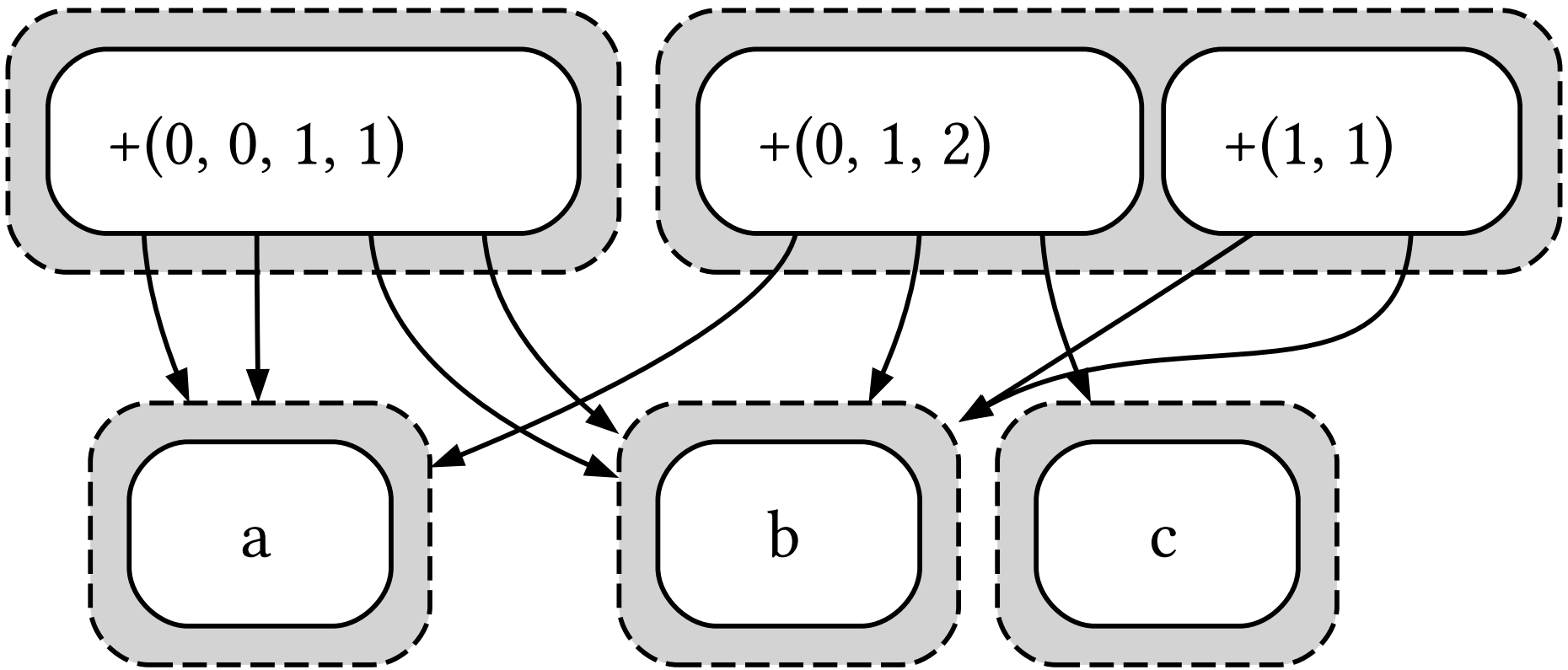


Figure 10: Starting graph

The AC word problem, brief example

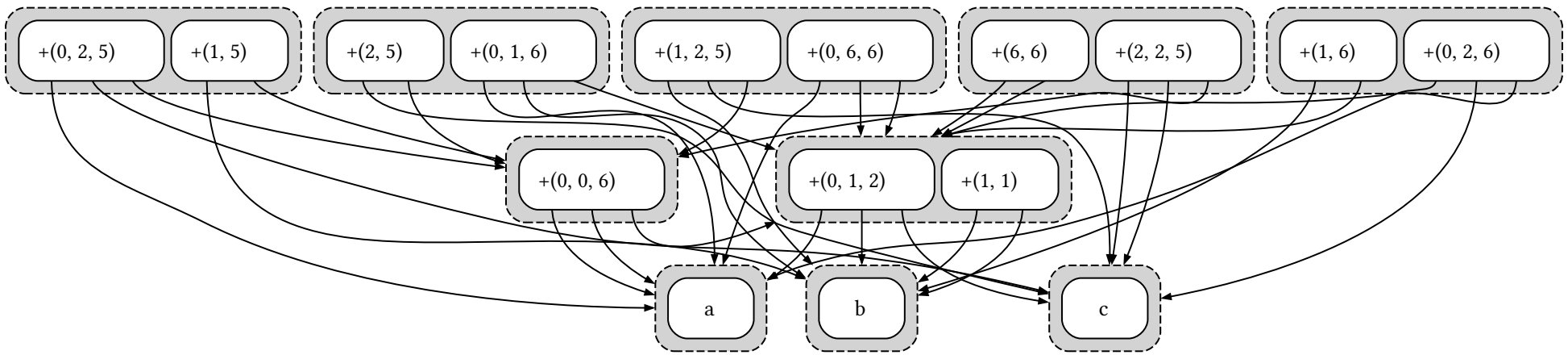


Figure 11: AC-completed graph

The AC word problem, the polynomial encoding

If $abc = def$, encode this as a polynomial equation $x_a x_b x_c - x_d x_e x_f = 0$.

This is not overkill, since we have the ESPACE lower bound…

The AC word problem, the polynomial encoding

If $abc = def$, encode this as a polynomial equation $x_a x_b x_c - x_d x_e x_f = 0$.

This is not overkill, since we have the ESPACE lower bound…

A set of AC equations form a *polynomial ideal*, and finding a reduced *basis* for this ideal solves the word problem. And this is also what algebraists have been doing for a long time: Buchberger's algorithm and Gröbner bases are standard tools

AC word problem example, as an ideal

$$(a + b) + c = e_1$$

$$b + (c + d) = e_2$$

$$\text{translate} \Rightarrow$$

$$abc - e_1 = 0$$

$$bcd - e_2 = 0$$

$$\Rightarrow$$

$$abcd - d(abc - e_1) = -e_1 d$$

$$abcd - a(bcd - e_2) = -e_2 a$$

AC word problem example, as an ideal

$$e_1 d - e_2 a = 0$$

Final basis:

$$abc - e_1 = 0$$

$$bcd - e_2 = 0$$

$$e_1 d - e_2 a = 0$$

Buchberger etc

Buchberger's algorithm, and the refined F_4 and F_5 variants, are what we would like to use.

How to integrate their term-ordering fiddlyness with our flattened E-class system?

Well, Tiwari solved it in his dissertation sections on congruence closure mod AC.

(It isn't that hard)

It turns out E-graphs have yet to fully absorb all the insights of the term-rewriting world.

Buchberger again

We can solve not just AC, but rings, polynomials, ACU, etc, with the same techniques.

Better than Buchberger

Abelian groups (aka ACUN) can be solved with Hermite normal form. Polynomial time! Vector spaces over a field can be solved with Gaussian elimination, which is $O\left(n^{2.\text{something}}\right)$

Matching modulo AC

I have ideas, but you will probably have to wait until EGRAPHS 2026 for the full story :)

Integrating theories and E-graphs: design choices

There are two ways to think about adding an AC-solver to an E-graph.

1. 'Integrated': Treat +-nodes as any other E-graph node, and scan for critical pairs between them
2. 'Side-table': Don't store +-nodes in the normal E-graph at all. Instead, have a side table of AC-rewriting rules.
   - When you add e.g. $e_1 + e_2$ to the E-graph, create a new rewrite $e_1 + e_2 \to e_3$ for some fresh e_3
   - Whenever we discover a rewrite $e_j \to e_k$ in the AC-solver, feed it back to the original E-graph

Integrating theories and E-graphs

I chose option 2 for my current implementation. Indeed, I didn't even think too hard about the tradeoff, as it seemed obvious that these are equivalent in some sense.

Not entirely equivalent methods

However, there is one advantage of the 'side-table' approach that might be accidentally missed in the integrated approach.

In the side-table approach, if we build the rewrite $e_1 + e_2 \to e_3$ and we have an existing rule $e_1 + e_2 + e_2 \to e_4$, we can compress the existing rule down to $e_2 + e_3 \to e_4$.

This is a standard step in textbook Buchberger's algorithm.

Not entirely equivalent methods

Just finding critical pairs is *semantically* correct, but misses this compression opportunity if you aren't paying attention.

To make the integrated method equivalent, we need to keep some ordering between +-nodes in the same class and do some destructive rewriting of the E-graph.

See my thesis...

I hope to go a bit more into depth about this design choice in my thesis.

Important sources

Decision Procedures in Automated Deduction: Ashish Tiwari's disseration

A Modular Associative Commutative (AC) Congruence Closure Algorithm: Deepak Kapur

MODULARITY AND COMBINATION OF ASSOCIATIVE COMMUTATIVE CONGRUENCE CLOSURE ALGORITHMS ENRICHED WITH SEMANTIC PROPERTIES: Deepak Kapur

Congruence Closure Modulo Associativity and Commutativity: L. Bachmair, I. V. Ramakrishnan, A. Tiwari, and L. Vigneron

The complexity of the word problems for commutative semigroups and polynomial ideals: Mayr and Meyer